TECHNICAL & STRATEGIC MEMO

Building a macOS Computer-Use RL Eval Fleet

Architecting 100 Apple-silicon machines into reinforcement-learning environments — and whether the resulting asset is worth anything to a lab that has a model.

Date	August 7, 2026
Scope	Fleet architecture · Eval integrity · Market positioning
Hardware	100 × MacBook / Mac mini / Mac Studio

CONTENTS

1. Fleet Architecture for macOS Computer-Use RL

1. The three constraints that drive every decision
2. Fleet partition
3. The individual machine
4. The fleet layer
5. Eval integrity
6. Build order

2. Is the Environment Layer Valuable Without a Model?

1. The reframe: verifiers, not VMs

2. Why macOS is defensible — and the honest risk
3. Who buys, and how to sell it
4. Economics and the 90-day proof
5. What kills it

PART ONE

Fleet Architecture for macOS Computer-Use RL

“I have 100 MacBooks / Mac minis / Mac Studios at my disposal. How would I set up this fleet to produce macOS computer-use RL evals and environments — at the individual machine level and at the whole-fleet level?”

The fleet design is driven by three hard macOS-specific constraints that don't exist on Linux, and everything else falls out of them:

> THE THREE CONSTRAINTS
>
> 1. **Apple's SLA caps you at 2 macOS VMs per Mac you own.** Capacity scales with *host count*, not host size. 100 Mac minis beat 10 Mac Studios for env slots.
> 2. **TCC (Screen Recording / Accessibility / Automation) will block synthetic input and capture** unless you pre-authorize it — which means either MDM PPPC profiles on bare metal, or SIP-disabled VMs where you write `TCC.db` directly.
> 3. **Apple-silicon VMs can't do everything bare metal can** (Secure Enclave, camera, Bluetooth, Touch ID, historically iCloud/App Store). So the fleet has to be *partitioned*, not uniform.

Fleet partition (100 machines)

POOL	COUNT	ROLE
VM hosts	70 Mac minis (M4/M4 Pro, 32–64 GB, 1–2 TB)	140 concurrent VM env slots. The bulk of throughput.
Bare-metal envs	20 minis	1 env each. Tasks needing camera / Bluetooth / Touch ID / App Store / real Continuity. Reset via MDM re-provision (slow — reserve for the ~10% of tasks that need it).
Infra	5 (Studios if you have them)	OCI image registry, control plane, artifact sink, record/replay proxy, local policy inference if wanted.
Human / authoring	MacBooks	Demo capture, task authoring, “nurse” Macs running Apple Configurator to revive bricked nodes, canary hosts for new macOS builds.

MacBooks are the worst rack citizens (battery swelling on permanent AC, lid and thermal management) — don't burn them as env slots. They're genuinely good as demonstration-capture stations, because human demos recorded through *the same* observation/action stack are what bootstrap SFT and define your human baseline.

The individual machine

Host layer (the Mac itself)

Runs no environment. It is a hypervisor and supervisor:

- **Hypervisor:** Virtualization.framework, driven by `tart`, `lume`, or ~500 lines of your own Swift. (Check `tart`'s Fair Source license at 100 nodes; `lume` / `macosvm` are permissive alternatives.)
- **Base images as OCI artifacts** pulled from a local registry (Harbor / zot), cached on disk. Per-episode VM creation is an **APFS clone** of the base — near-instant, block-shared, so 10 clones ≈ base + deltas.
- **A host agent** (~1k lines) that registers with the control plane, reports capacity and health, leases episodes, spawns and reaps VMs, GCs clones, and force-kills on deadline.

The reset primitive — the most important design choice

Don't boot a fresh VM each episode. Use `VZVirtualMachine.saveMachineState(to:)` (macOS 14+): boot the base image once, log in, launch the apps, let everything settle, then *suspend*. Every episode resumes from a byte-identical RAM + disk image in **1–3 seconds**.

Booting gives you ~30–45 s resets and non-identical state (login items race, Spotlight, notification timing). Suspend-restore gives you sub-3-second resets *and* the strongest determinism guarantee available on macOS. It is the single biggest lever on both throughput and eval reproducibility.

In-guest env agent (the piece you must build)

A small Swift daemon inside the guest exposing HTTP/WebSocket. Keep it *in-guest*, not host-side screen scraping, so the identical binary works on bare metal and in VMs:

```
GET  /screenshot    ScreenCaptureKit (SCStream) → JPEG/HEVC frame, ~5-16ms
GET  /axtree        AXUIElement walk → roles, labels, frames (grounding + cheap verifier)
POST /act           CGEventPost(kCGHIDEventTap): move/click/drag/scroll/key/type
GET  /quiesce       frame-delta + CPU-idle heuristic: "UI has settled"
POST /exec          shell/JXA, for setup + verify only (never exposed to the policy)
GET  /health
```

`/quiesce` matters more than people expect. A fixed `sleep(2)` after every action is how you turn a 3-minute episode into a 10-minute one *and* still get flakes.

Determinism checklist baked into the base image

This is where eval quality is won or lost. Bake into the golden image, then suspend:

- SIP off (recovery boot in the VM) → write `TCC.db` directly to pre-grant Accessibility, Screen Recording, and Automation. On bare metal, MDM PPPC profiles instead.
- Software Update pinned and disabled. **A silent macOS update changes the pixel distribution of every eval you've ever run.** Pin the OS version in the image tag.

- Fixed resolution and scale factor, fixed wallpaper, menu bar clock without seconds, fixed locale/timezone, `systemsetup -setusingnetworktime off` with an injected date.
- Notifications off, Focus off, Spotlight indexing off, Siri suggestions off, animations off (`NSAutomaticWindowAnimationsEnabled=false`, `reduceMotion`).
- Auto-login, no screensaver, no sleep.
- **Network via a central record/replay proxy** (mitmproxy reverse mode with recorded flows). Live internet is the largest single source of eval nondeterminism — a changed webpage silently breaks a task and you'll blame the model.

Task + verifier format

Keep it declarative so tasks are data, not code:

```
id: mail-forward-attachment-003
image: sonoma-14.6-office@sha256:...
setup:   scripts/seed_mailbox.sh      # runs in guest pre-suspend-fork
instruction: "Forward the invoice from Acme to accounting@… with subject 'FY25 Q3'"
verify:  scripts/check_sent_items.py  # runs in guest post-episode → {pass, partial, evidence}
timeout: 300s
```

Verify **programmatically against state** — app SQLite stores, filesystem, the proxy's request log — not against pixels. Model-judges over the trajectory are a fallback for genuinely subjective tasks, not the default. Every verifier should also emit *evidence* so you can audit a score six months later.

Per-machine latency budget (realistic)

STAGE	TIME
Screenshot capture + encode	10–25 ms
Transport to policy	5–50 ms
Policy inference	1–10 s ← dominates
Action injection	< 5 ms
UI settle (`/quiesce`)	200 ms – 3 s

The Mac is never your bottleneck. Design around the inference latency.

The fleet layer

Control plane

Do **not** put Kubernetes on the Macs. Two workable options:

- **HashiCorp Nomad** with `raw_exec` on each host — genuinely good macOS support, one job per episode, far lighter than k8s. This is the default recommendation.
- **Custom scheduler + host agents** (~1.5k lines) — lease / heartbeat / timeout, which is what most teams converge on anyway once they need image-affinity and warm-pool awareness.

Responsibilities: image registry and version pinning, task registry, episode scheduling (task → required image → host with that image warm), leases with deadlines, artifact sink, host quarantine, rolling image rollout with canaries.

Env API — standardize it and never let it leak Mac-ness

One Gym-shaped interface over HTTP/gRPC: `reset(task_id, seed) → obs`, `step(action) → obs, reward, done, info`. Trainers should not know whether the box is a Mac VM, a Linux container, or an Android emulator. Use a computer-use action schema close to the public ones (OSWorld / WindowsAgentArena / macOSWorld) so you can port existing task suites in and publish results that are comparable out.

Async, not lockstep

200 slots × multi-second steps means **synchronous batched rollouts will stall on your slowest Mac every single step.** Run actor–learner separation: envs pull actions from a central batched inference server, trajectories land in a replay buffer, the learner consumes asynchronously. Use per-episode deadlines with straggler abandonment, and count abandoned episodes as a fleet health metric — not as task failures.

Networking & storage

- Wired 1–10 GbE, VLAN per pool, DHCP reservations. Guests NAT'd behind the host, egress **allowlisted** through the record/replay proxy. Assume the guest is hostile: a policy with a mouse and keyboard will eventually find your control plane if it's reachable. It must not be reachable.
- Encode trajectories as **HEVC via VideoToolbox on the host** (hardware encoder, free), not PNG sequences. ~10 MB/episode instead of ~80 MB.
- Lifecycle policy: metadata + verifier evidence forever; video 30 days; full frames only for failed or flagged episodes.

Capacity math

```
90 env hosts × 2 VMs            = 180 concurrent slots
median episode 5 min (3s reset) → 2,160 episodes/hour
                                ≈ 50k episodes/day
at ~150 steps/episode           ≈ 7.5M env steps/day
storage @ ~12MB/episode         ≈ 600 GB/day
power @ ~70W/mini under load    ≈ 7 kW + cooling
```

Ops realities at 100 nodes

- **Smart PDUs are not optional.** Macs hang in states no software can recover; you will need remote hard power-cycle. Budget for it up front.

- Apple-silicon revive/restore needs a *second* Mac over USB-C (Apple Configurator / `cfgutil`). One nurse Mac per rack.
- MDM (Nanomdm / MicroMDM if you want open source; Jamf / Kandji if you want it to just work) for PPPC profiles, update deferral, and remote wipe on bare-metal nodes.
- APFS clones leak disk fast. Run a GC reaper on every host, plus a disk-pressure health check that quarantines the node before it wedges.

Eval integrity — easy to skip, expensive to retrofit

- **Version everything as a content hash:** `(image_digest, task_version, harness_version)`. A score is only comparable to another score with the same triple. Publish the triple with every number.
- **Golden replays, nightly.** Take a known-good scripted trajectory per task and re-run it across the fleet. Any task whose scripted solution doesn't pass 100% is nondeterministic — quarantine it. Any *host* with an elevated golden-replay failure rate is sick — quarantine it. This one practice is the main thing separating a real eval fleet from a demo that looked great once.
- **Track a flake budget per task** and refuse to admit tasks above ~2% into the eval split.
- **Strict train/eval task separation**, and treat eval task instructions as secrets — they leak into training data through trajectory logs faster than you'd think.
- Measure more than success rate: steps-to-success, wall-clock, per-app breakdown, **harmful-action rate** (files deleted, mail sent, settings changed), and recoverability after a wrong action.

Build order

Weeks 1–3 (2 Macs). In-guest env agent + golden base image + suspend/resume reset + 10 hand-written tasks with programmatic verifiers. Prove a single machine can run 100 clean episodes back-to-back with zero drift.

Weeks 4–6 (10 Macs). Image registry, host agent, Nomad or your scheduler, artifact sink, env API. Prove the throughput math holds linearly.

Weeks 7–10 (100 Macs). PDUs, MDM, monitoring, golden replays, GC, quarantine. Then scale the *task* pipeline — by month three the bottleneck is never machines, it's authored tasks with trustworthy verifiers. Plan for a task-authoring UI that turns a recorded human demo into a verifier skeleton, and treat LLM-proposed tasks as candidates requiring human confirmation, not as ground truth.

> PRIOR ART WORTH READING FIRST
>
> **OSWorld**, **WindowsAgentArena**, **macOSWorld**, and **trycua/cua** — the last of which already ships Virtualization.framework macOS sandboxes for computer-use agents, and may be a shortcut for weeks 1–3.

PART TWO

Is the Environment Layer Valuable Without a Model?

“What if I don't have a model to train, but I just design the tasks and orchestrate the task environments? Is that valuable to someone else who does have a model they want to do Mac computer-use RL for?”

> Yes — but not for the reason you'd expect, and the valuable part isn't the fleet.

The reframe

Orchestration is the commodity. Any lab with three good infra engineers rebuilds your control plane in a quarter, and they'd rather own it than depend on a vendor. If you pitch “I have 100 Macs and a scheduler,” you're selling a rounding error on someone's compute budget.

> THE ASSET
>
> **The verified task corpus.** Tasks with programmatic verifiers that are correct, flake-tested, versioned, and non-gameable. That is slow, taste-dependent, judgment-heavy work that does not compress with more engineers, and it is precisely what labs cannot buy off a shelf. The fleet is the factory; the verifiers are the product.
>
> Second asset, close behind: **trajectory data** — especially human demonstrations and dense failure traces on the same observation/action stack the tasks run on. Labs have plenty of successful-looking rollouts and very little well-labeled failure.

Why macOS specifically is a defensible niche

The constraints from Part One are the moat, not the obstacle:

- **You must own Apple hardware.** No containers, no Linux VMs, no elastic cloud. AWS EC2 Mac has a 24-hour minimum dedicated-host allocation (~$470/host/month), so 100 hosts is ~$47k/month rented versus ~$150–250k capex owned, amortizing to roughly $4–7k/month over three years. Owning is ~5–8× cheaper, and *nobody can burst past you on demand*. That's an unusual property in AI infra.
- **The 2-VM SLA cap** means capacity is bounded by machines-owned. Capital, not code.
- **TCC, MDM, PPPC, Apple Configurator revival, PDU power-cycling** — the grubby ops knowledge is a real six-month tax on any entrant.

Compare: browser environments are Playwright in a container, infinitely scalable, already commoditized by a dozen vendors. Linux desktop has OSWorld. **macOS is the underserved one, for structural reasons that persist.**

The honest risk

Demand for macOS computer-use is *thinner* than for browser or coding. The pull is real — every lab shipping a desktop app wants an agent that drives the user's actual Mac, and Apple's own platform work creates gravity — but it's a second-priority workstream at most labs today, not a first. You are betting that macOS graduates from “demo” to “funded roadmap” within roughly 18 months. That's a defensible bet, not a certainty, and you should size the business accordingly rather than assuming a Scale-sized outcome.

Who actually buys, ranked

1. **Frontier labs shipping desktop agents.** Highest willingness to pay, longest sales cycle, enormous leverage over you, and they will in-house it the moment it becomes strategic. Plan for that ending.
2. **Well-funded computer-use startups.** Faster to close, will pay for evals before they can pay for training data, more likely to become a durable customer because building this themselves is genuinely off-strategy for them.
3. **Mac-native ISVs** (design, finance, creative, dev tools) who need to know whether agents can drive their app — and increasingly need to make sure they can. Smaller checks, far more of them, much less lumpy. This is the underrated segment and the reason the business survives if segment 1 stalls.

How to sell it — and one legal detail that decides the shape

Do not ship the task corpus or the base images. Two reasons.

The commercial one: once a lab has your tasks, you have no second sale, and your holdout split is contaminated the day it leaves your network.

The one people miss: **you cannot legally distribute a base image containing licensed third-party software.** Your most valuable environments are the ones with Office, Adobe CC, Xcode, real productivity apps — and redistributing those images violates every one of those licenses, plus macOS's own terms about running only on Apple hardware you own.

> THE PRODUCT SHAPE BOTH REASONS POINT TO
>
> **Sell managed capacity, not artifacts.** The lab points a policy endpoint at your fleet; your Macs run the episodes; you return trajectories, scores, and evidence. Tasks and images never leave. Licensing stays clean because the software runs on hardware you own. Recurring revenue instead of one-time.

Make integration cost approximately zero — an OSWorld/macOSWorld-compatible action schema and a Gym-shaped API, so a lab can be running against your fleet in an afternoon. Nobody evaluates a vendor that takes two weeks to plug in.

The 90-day proof

The standard playbook works here, and it's cheap:

1. **Build 50 macOS tasks with bulletproof verifiers.** Not 500 mediocre ones. Flake-test each to <2% with golden replays.
2. **Publish ~20 of them open**, with the harness. Establishes credibility and costs you almost nothing.
3. **Run a public macOS computer-use leaderboard** against every frontier model via API. That's a few thousand dollars of inference on hardware you already own.
4. **Publish the results.** Every model will score badly — GUI agents on real macOS apps are genuinely hard.

That leaderboard is the entire go-to-market. A lab seeing its model at 23% on a credible macOS benchmark has a problem, and you are the only person holding the other 480 tasks and the fleet to train against them. This is exactly how SWE-bench, OSWorld, and τ-bench turned into commercial relationships.

Rough economics

High-quality verified tasks price somewhere in the low hundreds of dollars each; complex multi-app ones with rich verifiers go higher. Managed eval capacity prices per-episode or as a capacity commit. A small team with 100 Macs, a couple thousand excellent macOS tasks, and two or three lab relationships is plausibly a several-million-per-year business and a credible acqui-hire target. It is not, by itself, a venture-scale outcome unless macOS agents become a major surface — treat the leaderboard as the option on that.

Two practical notes: put **consent and PII handling** around human demonstration capture from day one (you're recording screens), and never let a customer's policy reach your control plane — a model with a mouse in a VM you're selling access to is a security problem you own.

What kills it

Apple shipping a first-party macOS agent harness and eval suite; labs concluding browser-only is sufficient; or someone open-sourcing 2,000 good macOS tasks. The first two you can't control. The third you partly defuse by being the one who publishes the open subset and owns the benchmark's name.

> **BOTTOM LINE**
>
> The 100 Macs are table stakes and a modest capital moat. Spend your effort on verifier quality and one credible public benchmark, sell managed capacity rather than data, and target the startups before the frontier labs. The fleet is what lets you do the valuable thing — it isn't the valuable thing.